ALBA ONE (意思決定OS): Enterprise Decision Operating System

In-Depth Architectural, Security & Engineering Reference Across All 11 Phases

Organization: ALBACONNECT Inc. (Tokyo, Japan — ISO/IEC 27001 ISMS Certified)

Target Runtime Stack: Python 3.12 (`requires-python = ">=3.12"`), Pydantic v2, FastAPI, SQLAlchemy 2.0 + Alembic, LangGraph, FastMCP (22 Servers), PostgreSQL 16 (`pgvector` + Sudachi Japanese FTS), Pinecone, Cohere Rerank v4, AWS Bedrock Guardrails (Standard Tier), Next.js 15 App Router, Kubernetes (Helm + ArgoCD + Istio mTLS), and Terraform.

Executive Architectural Thesis

Enterprise decision-making in logistics, vessel berth allocation, workforce staffing, and machine learning model governance fails when organizations rely on unconstrained large language model (LLM) chat interfaces. Standard conversational agents conflate knowledge retrieval, regulatory arithmetic, option comparison, and final approval authority inside a single opaque generation pass. In regulated Japanese industrial operations—where compliance with the 2024 Logistics Overtime Cap (物流2024年問題, 960 hours/year driver limit), strict multi-tenant data boundaries, and Japanese Personal Information Protection Act (APPI / 個人情報保護法) controls are mandatory—such opacity is unacceptable.

ALBA ONE (`意思決定OS`) solves this problem through an eleven-phase architecture built around four non-negotiable engineering invariants:

1. **Deterministic Code Before LLM Prose:** All domain math (SLA feasibility, driver overtime premiums, vessel berth conflict detection, model release latency/cost trade-offs), multi-criteria option scoring, verdict ceiling gates, and post-decision outcome error metrics execute in pure, unit-tested Python before any language model writes narrative synthesis.
2. **Zero-Text Vector Store Invariant:** On-premise PostgreSQL remains the sole system of record for sensitive corporate text. Before any vector leaves the premises, Japanese and English PII is redacted or tokenized, and Pinecone receives only 1536-dimensional embeddings plus opaque metadata identifiers (`chunk_id`, `tenant_id`, `source_table`, `acl_tags`, `lang`)—never raw chunk text.
3. **Seven-Layer Defense-in-Depth Guardrails:** Every workflow traverses Input, Retriever, Tool/Outbound, Output, Human-in-the-Loop (HITL), Operational Budget, and Prompt-Hardening layers, pairing AWS Bedrock Guardrails Standard Tier (`apac.guardrail.v1:0`) with deterministic SQL, SSRF, and canary-leak filters.
4. **Strict Separation of AI Recommendation and Human Decision:** The Judge emits a bounded `RecommendationVerdict`; only a verified human approver (`requester_id != approver_id`) can record an `ApprovalAction`.

Verification Dimension	Implemented Artifact Scope	Verified Count
Phases Completed	Foundation through Post-Deployment Continuous Red Teaming (Phases 1–11)	11 / 11
Automated Unit, Contract & Meta Tests	Pure-logic unit tests, 22 MCP server contract suites, and red-team self-tests	196 Passed
Isolated FastMCP Tool Servers	13 internal enterprise servers + 9 external intelligence servers in `registry.yaml`	22 Servers
Bilingual Evaluation & Red-Team Corpus	Retriever (60), Chunking (6), MCP (352), Calculators/Judge/E2E (54), Safety/Red-Team (252)	724 Scenarios
Modular Code Hygiene	Every Python module across `apps/api`, `evals`, and `redteam` verified ≤ 300 lines	100% Compliant

Master Architecture Overview & Cross-Phase Verification

The ALBA ONE monorepo is structured as a strict uv workspace targeting Python 3.12 and a Next.js 15 TypeScript frontend. To guarantee that core business rules, security scorers, and regulatory calculators can be verified deterministically in any continuous integration environment without external network dependencies, every subsystem separates pure domain logic from framework glue. Figure 1 summarizes the distribution of the 196 automated unit, contract, and meta-tests alongside the 724 bilingual evaluation and adversarial scenarios across all eleven phases.

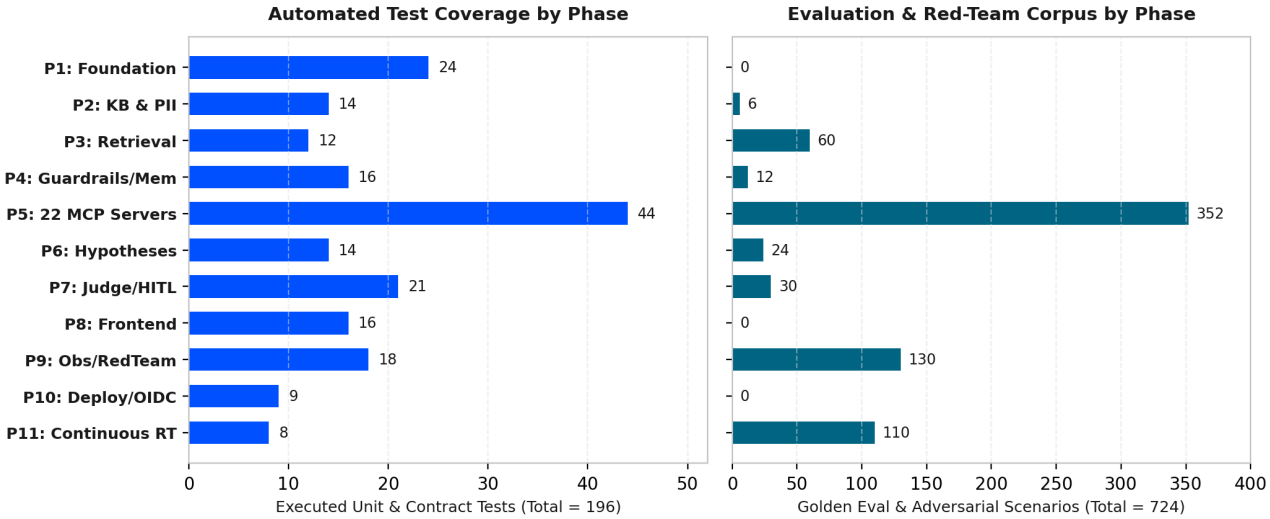


Figure 1: Distribution of the 196 executed unit/contract/meta-tests (left) and 724 bilingual golden evaluation and red-team scenarios (right) across Phases 1 through 11.

Phase	Core Subsystem & Primary Modules	Primary Architectural Guarantee	Key Verification Suite
Phase 1	Foundation (`core/`, `gateway/`, `cache/`, `db/`, `schemas/`)	Multi-tenant context isolation, swappable LLM gateway headers, bilingual errors, Redis Lua rate limiting	`test_config_and_pricing.py`, `test_tenancy_and_logging.py`, `test_cache_and_ratelimit.py`
Phase 2	Knowledge Base & Ingestion (`kb/`, `config/ontology_demo.yaml`)	Read-only DBA views (`v_kb_*`), JP/EN PII redaction, Sudachi FTS tokenization, text-free Pinecone indexing	`test_kb_extraction_and_pii.py`, `test_chunkers_and_fts.py`, `evals/chunking_compare.py`
Phase 3	Retrieval Pipeline & Eval (`retrieval/`, `evals/retriever_eval.py`)	Ambiguity clarification, bilingual query rewriting, RRF hybrid fusion, Cohere v4 reranking, CRAG grading	`test_retrieval_pipeline_and_eval.py`, `goldens/retriever.json` (60 JA/EN items)
Phase 4	Guardrails, Memory & Ops Controls (`guardrails/`, `memory/`)	7-layer defense pipeline, Bedrock Standard Tier (`apac.guardrail.v1:0`), HITL-confirmed Mem0 OSS memory	`test_guardrails_and_memory.py`, `apps/api/tests/memory/contract.py`
Phase 5	22 MCP Servers & Tool Evals (`mcp_servers/`, `mcp_client/`)	Declarative `registry.yaml`, pluggable web search, DNS-resolved SSRF protection, 22 per-server evals	`test_mcp_and_ssrf.py` + 22 `tests/mcp/<srv>/test_contract.py` suites
Phase 6	Hypothesis Generation (`agents/hypothesis/`, calculators)	4 deterministic domain calculators, 2-round shared evidence ledger, mandatory status-quo baseline `H0`	`test_calculators_and_units.py`, `test_hypothesis_validator_and_graph.py`

Phase	Core Subsystem & Primary Modules	Primary Architectural Guarantee	Key Verification Suite
Phase 7	Decision Judge, HITL & Outcomes (`agents/judge/`, `hitl/`, `outcomes/`)	Deterministic `ComparisonTable`, Verdict Ceiling Rules `a`–`g`, 16-state HITL outbox, steward write-back gate	`test_judge_comparison_and_gates.py`, `test_hitl_state_and_outbox.py`, `test_outcomes_and_writeback.py`
Phase 8	Frontend Workspace (`apps/web/`, `tailwind.config.ts`)	ALBACONNECT brand tokens (zero green), `httpOnly` cookie proxy, character-level JP diff, prod mock ban	`test_phase8_frontend_contracts.py`, `apps/web/tests/design-tokens.test.ts`
Phase 9	Observability & Red-Team Harness (`observability/`, `redteam/`)	Unified PostgreSQL `spans` table, versioned USD/JPY pricing, 11-family (110-scenario) red-team harness	`test_observability_and_ops.py`, `test_redteam_and_safety.py`
Phase 10	Deployment, OIDC & Hardening (`core/auth_oidc.py`, `deploy/`)	RS256/ES256 OIDC + JWKS anti-amplification, dual-secret S2S rotation, fail-closed boot, read-only containers	`test_phase10_auth_and_startup.py`, `ci/scripts/check_images.sh`
Phase 11	Continuous Post-Deploy Red Teaming (`redteam/isolation.py`, `compare.py`)	4-control target isolation, lying-target abort, Wilson 95% CI drift gates, blast-radius containment without auto kill switch	`test_phase11_post_deploy_redteam.py`

Phase 1: Foundation — Multi-Tenant Contexts, Swappable LLM Gateway, Bilingual Errors & Rate Limiting

Architectural Importance and Design Rationale

Phase 1 establishes the structural bedrock required before any retrieval or agentic workflow can execute safely. In an enterprise multi-tenant deployment, three failure modes commonly arise if foundational concerns are retrofitted late: (1) tenant identifiers are passed inconsistently across endpoints, allowing cross-tenant cache or database leaks; (2) direct SDK coupling to a single model vendor prevents rapid failover during upstream outages; and (3) missing environment variables fail silently at runtime only when a rare code path is triggered. Phase 1 eliminates all three risks at startup.

First, `apps/api/src/alba_one/core/config.py` and `core/env_validator.py` implement strict startup validation via `pydantic-settings`. If any required configuration variable is absent, `get_settings()` raises a `MissingConfigurationError` naming every missing variable in a single human-readable message. Furthermore, `check_eval_judge_distinct(main_model, eval_judge_model)` verifies that the evaluation judge model (`claude-3-5-sonnet-20241022`) is distinct from the primary generation model (`gpt-4o-2024-08-06`) so self-preferential bias is prevented during automated evaluations.

Second, `apps/api/src/alba_one/gateway/routing.py` and `gateway/llm.py` (documented in `docs/adr/0001-llm-gateway.md`) decouple all LangChain `ChatOpenAI` invocations from upstream vendor lock-in. Every model call attaches a structured `GatewayCallMetadata` record containing `tenant_id`, `task`, `prompt_version`, and `trace_id`. Switching between `portkey`, `bifrost`, `tensorzero`, and `mesh` requires changing only `GATEWAY_PROVIDER` and `GATEWAY_BASE_URL` in configuration.

Concrete Implementation & Execution Examples

When the hypothesis generator invokes the LLM gateway for tenant `tenant_alba_demo`, `build_gateway_headers()` constructs both vendor-neutral ALBA audit headers and provider-specific routing metadata while deduplicating fallback chains via `resolve_fallback_chain("gpt-4o-2024-08-06", ["gpt-4o-mini-2024-07-18", "gpt-4o-2024-08-06"])`:

```
{"x-alba-tenant-id": "tenant_alba_demo", "x-alba-task": "hypothesis_generation", "x-alba-prompt-version":
"hyp_v1.2", "x-alba-trace-id": "tr_001", "x-portkey-metadata": "
{"tenant_id":"tenant_alba_demo","task":"hypothesis_generation","prompt_version":"hyp_v1.2"}"}
```

Similarly, `apps/api/src/alba_one/cache/keys.py` and `cache/llm_cache.py` enforce tenant-prefixed cache keys (`alba:llm_cache:{tenant_id}:{model}:{prompt_version}:{sha256}`) so two tenants submitting identical prompts can never hit each other's

cached responses. Rate limiting in `cache/rate_limit.py` uses an atomic Redis token-bucket algorithm across three independent scopes (tenant RPM: 300, user RPM: 60, tool RPM: 120), raising a bilingual `AlbaError(code="RATE_LIMIT_EXCEEDED", status_code=429)` rendered in both Japanese (`"レート制限を超過しました。12秒後に再試行してください。"`) and English (`"Rate limit exceeded. Please retry after 12 seconds."`) with the request's `trace_id`.

Phase 2: Knowledge Base & Ingestion — Read-Only SQL Views, Japanese PII Scrubbing & Hybrid Indexing

Architectural Importance and Design Rationale

Phase 2 bridges the customer's on-premise operational PostgreSQL database with ALBA ONE's retrieval engine without ever exposing raw database credentials, unapproved tables, or Personal Identifiable Information (PII) to external cloud services. The ingestion pipeline is governed by a declarative per-tenant ontology (`config/ontology_demo.yaml` and `kb/ontology.py`) that maps eight core business entities—`company`, `department`, `staff`, `vessel`, `route`, `delivery_order`, `equipment`, and `model_release`—to explicit permission tags (`acl_tags`) and decision-critical attributes.

To enforce least privilege at the database boundary (documented in `docs/KB_EXTRACTION.md`), `kb/extractors.py` refuses to query base tables directly. All SQL extraction executes strictly through five DBA-controlled read-only views defined in `infra/seed/02_kb_readonly_views.sql`: `v_kb_company_departments`, `v_kb_delivery_routes`, `v_kb_vessels_berths`, `v_kb_equipment_status`, and `v_kb_model_releases`. Any attempt to query a table outside `ALLOWED_EXTRACTION_VIEWS` is rejected deterministically before reaching the database driver.

Japanese PII Redaction, Morphological Tokenization & Chunking

Before any extracted document is chunked or sent to the embedding endpoint, `kb/pii.py` (`PIIRedactor`) scrubs or HMAC-tokenizes sensitive identifiers across both Japanese and English formats:

- **Japanese Individual Number (マイナンバー / My Number):** 12-digit patterns (□\d{4}[-\s]?\d{4}[-\s]?\d{4}□) are replaced with `[REDACTED_MYNUMBER]`.
- **Japanese Phone & Postal Codes:** Domestic mobile/landline numbers (`090-1234-5678`, `03-5555-0199`) and postal codes (`〒100-0005`) are scrubbed to `[REDACTED_PHONE]` and `[REDACTED_POSTAL]`.
- **Configurable Named-Entity Dictionary & Emails:** Personal names from the tenant's HR roster and email addresses are redacted or deterministically tokenized per tenant salt so relational links remain intact without leaking identities.

For Japanese full-text search (documented in `docs/adr/0002-japanese-fts-tokenizer.md` and implemented in `kb/fts.py`), whitespace splitting fails because Japanese sentences lack word delimiters, and simple character bigrams split compound logistics nouns inconsistently. ALBA ONE combines **SudachiPy** dual-mode morphological tokenization—indexing both `SplitMode.C` (long compound nouns such as 東京大井ふ頭 and 燃料サーチャージ) and `SplitMode.A` (constituent sub-tokens)—with PostgreSQL `pg_bigm` fallback for exact alphanumeric model/route codes (`RT-KNT-101`, `VSL-kanto-02`).

Concrete Example: Text-Free Pinecone Upsert & Chunking Evaluation

`kb/chunkers.py` implements four selectable chunking strategies: `fixed`, `recursive`, `semantic`, and `parent_child` (the default, pairing fine-grained child chunks for dense vector precision with parent context windows for LLM synthesis). When `kb/indexer.py` indexes route `RT-KNT-101` (Tokyo Oi Pier to Yokohama Honmoku, SLA 4.0 hours, assigned truck `TRK-104`), it writes the full bilingual text into the on-premise PostgreSQL store while upserting **only** the following text-free metadata dictionary to Pinecone:

```
{"chunk_id": "chk_rt_knt_101_c01", "tenant_id": "tenant_alba_demo", "source_table": "v_kb_delivery_routes",
"source_pk": "RT-KNT-101", "entity_ids": ["RT-KNT-101", "TRK-104"], "acl_tags": ["role:analyst",
"role:approver", "dept:logistics"], "lang": "ja"}
```

An explicit invariant check in `kb/indexer.py` asserts that neither `"text"` nor `"content"` keys ever appear in the Pinecone payload. The companion evaluation script `evals/chunking_compare.py` benchmarks all four strategies on `goldens/chunking_smoke.json`, confirming that `parent_child` achieves the highest entity-preservation and citation-completeness score.

Phase 3: Retrieval Pipeline & Retriever Eval — Clarification, Bilingual Rewriting, Hybrid RRF & CRAG

Architectural Importance and Design Rationale

Even with a well-indexed knowledge base, naive single-query vector search degrades sharply when users ask underspecified questions or query in Japanese while technical specifications (such as ML model cards or international vessel manifests) are stored in English. Phase 3 implements a seven-stage Corrective RAG (CRAG) pipeline in `apps/api/src/alba_one/retrieval/` that resolves ambiguity upfront, bridges the Japanese–English lexical gap, fuses dense and sparse signals, reranks candidates with cross-encoders, and self-corrects when retrieval confidence is low.

Stage	Module	Core Mechanism & Failure Policy	Concrete Example Input / Output
1. Ambiguity Clarifier	`retrieval/clarifier.py`	Detects underspecified queries missing route, timeframe, or target entity; returns structured `ClarificationQuestion` options (or auto-selects defaults in `--non-interactive` mode).	Query `"配送の遅延リスクは？"` → asks which corridor (`RT-KNT-101 京浜` vs. `RT-KNS-204 阪神`) and date window.
2. Bilingual Query Rewriter	`retrieval/query_rewriter.py`	Expands user query into `QUERY_REWRITE_COUNT=4` cross-lingual queries (`ja_queries` + `en_queries`) using `i18n/glossary.json` domain synonyms and extracts `entity_filters`.	`"東京大井ふ頭から横浜本牧への配送ルートRT-KNT-101の割当車両とSLA時間は？"` → extracts `["RT-KNT-101"]` + EN query `"Tokyo Oi Pier to Yokohama Honmoku route RT-KNT-101 assigned vehicle and SLA hours"`.
3. Query Router	`retrieval/router.py`	Routes intent across `kb_hybrid`, `sql_structured`, `multimodal`, and `external_web` based on entity patterns and domain cues.	Route/vessel IDs route to `kb_hybrid` + `sql_structured`; market/weather cues enable `external_web`.
4. Hybrid Search + RRF	`retrieval/hybrid.py`	Executes Pinecone dense vector search and Postgres Sudachi/`pg_bigm` FTS in parallel, enforcing `tenant_id` and user `acl_tags` in both stores before fusing via Reciprocal Rank Fusion ($RRF(d) = \sum 1 / (60 + rank_i(d))$).	Chunk `chk_rt_knt_101_c01` ranked #1 in FTS (exact code match) and #2 in dense search receives top fused score.
5. Dual-Mode Reranker	`retrieval/reranker.py`	Supports `RERANK_PROVIDER=cohere_direct` (Cohere v2 API with `rerank-v4.0-pro`) and `RERANK_PROVIDER=gateway` (forwarding `RERANK_MODEL` unchanged). Falls back to RRF order if upstream times out.	Re-scores top 20 RRF candidates down to `RERANK_TOP_K=5` high-precision chunks.
6. Relevance Grader	`retrieval/grader.py`	Scores each chunk against `RELEVANCE_THRESHOLD=0.35` and checks entity coverage.	Chunks scoring < 0.35 are dropped before synthesis; if all drop, triggers CRAG loop.
7. Corrective RAG Loop	`retrieval/corrective.py`	Broadens query terms and optionally invokes governed external search (`MCPExternalSearchAdapter`) when internal KB coverage is insufficient.	Attaches external regulatory or marine weather evidence with verified source URLs.

Retriever Ablation Evaluation (`evals/retriever_eval.py`)

The evaluation harness in `evals/retriever_eval.py` runs against `goldens/retriever.json` (60 bilingual golden items: 30 Japanese and 30 English, tagged with `status: "draft"` and gated by `--allow-draft` so unreviewed synthetic benchmarks are never misrepresented as production sign-off). Running with `--compare` executes an automated four-way ablation across (1) Dense-only, (2) FTS-only, (3) Hybrid RRF without rerank, and (4) Full Hybrid RRF + Rerank + Query Rewriting, computing `Recall@5`, `Precision@5`, `nDCG@10`, and `MRR` per language.

Phase 4: Guardrails, Memory & Operational Controls — 7-Layer Defense, Bedrock Standard Tier & Mem0 OSS

Bedrock Guardrails Standard Tier & Cross-Region APAC Data Residency

A critical architectural finding documented in `docs/GUARDRAIL_COVERAGE.md`, `docs/SECURITY.md`, and `docs/DATA_FLOWS.md` governs Japanese guardrail deployment on AWS. Amazon Bedrock Guardrails' **Classic tier** supports only English, French, and Spanish; **Japanese (`ja`) is supported only on the Standard tier** (`BEDROCK_GUARDRAIL_TIER=STANDARD`). Furthermore, AWS requires the Standard tier in `ap-northeast-1` (Tokyo) to use the cross-Region APAC inference profile (`BEDROCK_GUARDRAIL_CROSS_REGION_PROFILE=apac.guardrail.v1:0`), which may route guardrail evaluation payloads across Tokyo, Osaka, Seoul, Mumbai, Singapore, and Sydney. Accordingly, `infra/bedrock/create_guardrail.py` explicitly provisions the Standard tier and fails closed with a clear error if `BEDROCK_GUARDRAIL_TIER=CLASSIC` is combined with Japanese policies.

The Seven-Layer Guardrail Pipeline (`apps/api/src/alba_one/guardrails/`)

1. **Layer 1 — Input Guardrail (`input_layer.py`):** Runs PII redaction, secret pattern blocking (API keys, private keys, bearer tokens), prompt-injection heuristics (Japanese & English override patterns), and Bedrock Standard-tier input evaluation. If Bedrock is unreachable, Layer 1 **fails closed** (`docs/FAILURE_POLICY.md`).
2. **Layer 2 — Retriever Guardrail (`retriever_layer.py`):** Enforces strict `tenant_id` and `acl_tags` filtering on every retrieved chunk, neutralizes indirect prompt-injection directives embedded inside documents, and wraps chunk bodies in `<retrieved_data chunk_id="...">` XML delimiters.
3. **Layer 3 — Tool & Outbound Query Guardrail (`tool_layer.py` & `outbound_query.py`):** Validates tool schemas, enforces read-only SQL view allowlists, blocks SSRF targets, and checks outbound external queries against the tenant's `sensitive_terms` catalog (blocking internal codenames like `FICTIONAL-CODE-KAGUYA-架空` under `OUTBOUND_QUERY_POLICY=block`).
4. **Layer 4 — Output Guardrail (`output_layer.py`):** Scans generated recommendations for leaked PII, secrets, markdown image beacons (`![...](https://...)`), or XSS tags (`<script>`), and verifies contextual grounding against the cited evidence ledger.
5. **Layer 5 — HITL Guardrail (`hitl_layer.py`):** Blocks any write-class or side-effecting tool call that lacks a verified, non-expired human `approval_id` from a distinct approver.
6. **Layer 6 — Operational Budget & Kill-Switch Guardrail (`ops_layer.py`):** Enforces `OPS_KILL_SWITCH`, per-request token ceiling (`32,000`), per-request cost ceiling (`$1.50`), daily tenant budget (`$250.00`), max LangGraph steps (`15`), and max tool calls per run (`10`).
7. **Layer 7 — System Prompt Hardening (`prompt_layer.py` & `prompts/decision_system/v1.0.md`):** Sandwich-wraps untrusted data blocks with explicit bilingual non-instruction reminders so data inside `<retrieved_data>` or `<tool_output>` is never executed as commands.

Self-Hosted Mem0 OSS Long-Term Memory (`memory/long_term.py` & `ADR-0003`)

To prevent sensitive corporate facts from being sent to Mem0's hosted SaaS cloud, ALBA ONE uses self-hosted `mem0ai` OSS backed by our on-premise PostgreSQL (`pgvector`) store and routes Mem0's internal fact-extraction LLM calls through our configured LLM Gateway. Every memory entry is scoped by `(tenant_id, scope_id, category)`, scrubbed by `PIIRedactor` before persistence, and requires explicit two-step confirmation (`propose_memory` → `confirm_memory` by the user or an approver) before becoming active in retrieval.

Phase 5: 22 FastMCP Servers with Per-Tool Evals & SSRF-Hardened Web Fetching

Single-Source Declarative Registry (`mcp_servers/registry.yaml`)

Phase 5 implements 22 isolated Model Context Protocol (MCP) servers—13 internal enterprise servers and 9 external intelligence servers—governed by a single declarative catalog in `apps/api/src/alba_one/mcp_servers/registry.yaml`. A parameterized multi-stage Dockerfile (`apps/api/Dockerfile.mcp`) builds each server with only its dedicated dependency extra from `apps/api/pyproject.toml` (for example, `--extra mcp-knowledge-base` or `--extra mcp-web-search`), while `infra/docker-compose.yml` uses a shared `x-mcp-service` YAML anchor and Compose profiles (`core`, `domain`, `external`, `all`) so developers can launch lightweight subsets locally.

Class	MCP Server Name (Port)	Primary Tools Exposed	Security & Governance Controls
Internal (Core & Data)	`knowledge_base` (8101), `sql_readonly` (8102), `departments` (8103), `approvals` (8104), `decision_history` (8107), `memory` (8108)	`search_kb`, `query_readonly_view`, `list_departments`, `check_approval_status`, `search_past_decisions`, `propose_memory`	Tenant ACL filter, `ALLOWED_EXTRACTION_VIEWS` SQL guard, HITL approval token verification on write tools
Internal (Domain & Ops)	`experiment_tracking` (8105), `benchmark_registry` (8106), `delivery_planning` (8109), `staffing` (8110), `vessel_allocation` (8111), `document_extraction` (8112), `deployment_status` (8113)	`calculate_route_sla_and_cost`, `calculate_driver_overtime`, `calculate_berth_schedule`, `calculate_release_tradeoffs`, `extract_document_tables`	Deterministic physical/financial unit validation, Japanese 2024 Logistics overtime cap checks, read-only MLflow/deployment queries
External Intelligence	`web_search` (8114), `news_search` (8115), `competitor_intel` (8116), `model_benchmarks` (8117), `papers` (8118), `github_releases` (8119), `market_data` (8120), `weather_marine` (8121), `regulatory_jp` (8122)	`web_search`, `fetch_page`, `search_industry_news`, `get_bunker_fuel_index`, `get_port_weather_forecast`, `lookup_mlit_regulation`	Outbound sensitive-term blocker (`outbound_query.py`), DNS-resolved SSRF guard (`fetcher.py`), domain allowlist/denylist, egress proxy routing

Pluggable Web Search & DNS-Rebinding-Resistant `fetch_page`

The web search subsystem (`mcp_servers/web_search/providers/`) defines a clean `WebSearchProvider` protocol with four swappable implementations: `tavily.py` (default), `brave.py`, `firecrawl.py`, and a deterministic offline `fake.py` provider reading bilingual fixtures from `goldens/fixtures/web_search/`. Page fetching is separated into `mcp_servers/web_search/fetcher.py` (`fetch_page(url)`), which resolves every hostname via DNS before connecting and blocks loopback (`127.0.0.0/8`, `::1`), RFC 1918 private networks (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), link-local/cloud metadata endpoints (`169.254.169.254`), and non-HTTP(S) schemes. Crucially, `fetch_page` disables automatic HTTP redirects and re-runs full DNS resolution and IP validation on every `301/302/307/308` redirect hop, defeating DNS-rebinding and open-redirect SSRF attacks. Every one of the 22 MCP servers has a dedicated contract test suite (`apps/api/tests/mcp/<server>/test_contract.py`) and a 16-item bilingual golden dataset (`goldens/mcp/<server>.json`, totaling 352 golden tool evaluations).

Phase 6: Hypothesis Generation — Deterministic Domain Calculators & Bounded 2-Round Evidence Ledger

Why Arithmetic Must Never Be Delegated to an LLM

When evaluating operational decisions—such as rerouting container freight from Tokyo Oi Pier, scheduling driver overtime during peak port congestion, allocating berth windows for a delayed vessel, or promoting a new LLM checkpoint—language models frequently hallucinate arithmetic or misapply statutory thresholds. Phase 6 solves this by implementing four pure-Python deterministic calculators in the domain MCP servers, backed by explicitly versioned regulatory parameters in `config/regulatory_params.yaml` (gated by `verified_by: TODO(owner)`):

- **Delivery SLA & Cost Calculator (`delivery_planning/calculators/sla_and_cost.py`):** Computes transit time ($t = d / v + t_{dwell} \times (1 + c_{congestion})$), fuel surcharge in JPY, and SLA breach margin in hours.
- **Workforce Overtime & 2024 Logistics Cap Calculator (`staffing/calculators/overtime.py`):** Enforces Japan's Labor Standards Act and 2024 driver overtime regulations (`max_driver_annual_overtime_hours: 960`, monthly standard cap `45h`, special clause `80h`, and mandatory `50%` statutory wage premium above `60h/month`), returning `compliance_status: "compliant" | "warning" | "breach"`.
- **Vessel Berth Conflict & Demurrage Calculator (`vessel_allocation/calculators/berth.py`):** Checks draft clearance (`vessel_draft_m + under_keel_clearance_m <= berth_depth_m`), crane capacity, tidal window overlap, and hourly demurrage penalty in USD/JPY.
- **ML Model Release Trade-Off Calculator (`experiment_tracking/calculators/release_tradeoffs.py`):** Evaluates p95 latency delta, cost per 1k requests delta, accuracy gain, and safety regression gates against production SLOs.

Bounded Two-Round Shared Gathering & Mandatory Status-Quo Baseline (`H0`)

Rather than allowing each hypothesis to spawn unbounded sub-agent tool loops, `agents/hypothesis/planner.py` and `graph.py` execute a bounded two-round gathering strategy writing into a single per-run `EvidenceLedger` (`agents/hypothesis/ledger.py`):

1. **Round 1 (Broad Shared Gathering, max `ROUND1_MAX_CALLS=6`):** Gathers core KB chunks and runs the relevant domain calculators in parallel via `asyncio.gather` with per-call timeouts (`TOOL_TIMEOUT_SECONDS=10.0`).
2. **Round 2 (Targeted Gap-Filling & Mandatory Counter-Evidence, max `ROUND2_MAX_CALLS=4`):** Explicitly queries for disconfirming evidence, failure history, and risk factors against the candidate options.

From the shared ledger, `agents/hypothesis/synthesizer.py` generates up to `MAX_OPTIONS=4` testable candidate hypotheses (`H1..Hn`) plus one **mandatory baseline hypothesis (`H0_baseline`: "Do nothing / maintain the current status quo")**. Finally, `agents/hypothesis/validator.py` verifies that every cited `ref_id` exists in the `EvidenceLedger` and that every numeric figure stated in a hypothesis matches a calculator output within rounding tolerance, rejecting ungrounded claims before Phase 7.

Phase 7: Decision Judge, 16-State HITL Workflow & Two-Stage Outcome Calibration Loop

Deterministic Comparison Matrix & Verdict Ceiling Rules (`a–g`)

Phase 7 strictly separates AI analysis from human authority: the Judge emits a `RecommendationVerdict`, whereas a human records an `ApprovalAction`. Before any LLM prose is generated, `agents/judge/comparison.py` builds a deterministic `ComparisonTable` across the domain criteria defined in `config/decision_criteria/{delivery_planning,staffing,vessel_allocation,model_release,default}.yaml`. Next, `agents/judge/gates.py` applies seven deterministic **Verdict Ceiling Rules (`a–g`)** that establish an upper bound on what the AI is allowed to recommend:

- **Rule a (Compliance / Safety Breach):** If the top option violates any regulatory or safety constraint (e.g., driver annual overtime > 960h or berth draft clearance breach), the recommendation ceiling is capped at `recommend_reject`.
- **Rule b (Low Evidence Confidence):** If maximum option confidence is `< 0.45`, the ceiling is capped at `needs_more_info`.
- **Rule c (Moderate Confidence / Missing Counter-Evidence):** If confidence is in `[0.45, 0.70)` or counter-evidence is incomplete, the ceiling is capped at `recommend_approve_with_conditions`.
- **Rule d–g (Unverified Regulatory/Weight Configs, High Risk, Close Margin, Baseline Superiority):** If `config/regulatory_params.yaml` or criterion weights still carry `TODO(owner)`, or if the status-quo baseline `H0` outperforms all action options on risk-adjusted score, the ceiling is restricted to `recommend_approve_with_conditions` or `recommend_reject`.

Crucially, if the LLM attempts to output `recommend_approve` when Rule c capped the ceiling at `recommend_approve_with_conditions`, `enforce_verdict_ceiling()` deterministically downgrades the verdict to the ceiling and records the override in the audit trail.

16-State HITL State Machine, Separation of Duties & Transactional Outbox

`agents/decision_state.py` and `agents/hitl/service.py` govern the lifecycle of every decision across 16 explicit states (including `awaiting_approval`, `approved`, `approved_with_conditions`, `rejected`, `needs_more_evidence`, and `expired`) with a hard cap of `MAX_EVIDENCE_LOOPS=2`. When a human submits `POST /api/v1/approvals/{decision_id}/decide`:

1. **Auth-Context Identity Binding:** `approver_user_id` is taken strictly from the verified authentication context (`UserContext`), never from the request body.
2. **Separation of Duties:** If `approver.user_id == decision.requester_user_id`, the service rejects the request with HTTP `403 Forbidden` (self-approval is impossible).
3. **Optimistic Concurrency & Stale-State Protection:** The request must supply the exact `recommendation_version_hash` currently active on the decision; a mismatch returns HTTP `409 Conflict`.
4. **Idempotency & Transactional Outbox (`agents/hitl/outbox.py`):** Requires an `Idempotency-Key` header and writes the approval record, append-only audit log, and LangGraph checkpoint resume event atomically.

Two-Stage Outcome Tracking & Anti-Contamination KB Write-Back (`agents/outcomes/service.py`)

After a decision is executed in the real world, ALBA ONE closes the feedback loop in two distinct stages so unverified post-mortems never pollute the knowledge base:

- **Stage 1 — Deterministic Outcome Recording (`POST /api/v1/decisions/{id}/outcomes`):** Computes exact predicted-vs-actual error metrics ($e_{abs} = |y_{actual} - y_{pred}|$, e_{pct}, and whether the outcome fell inside the predicted interval) in pure Python.
- **Stage 2 — Knowledge Steward Write-Back Gate:** Writing a distilled lesson back into the KB requires a separate user with the `knowledge_steward` role (`WRITEBACK_REQUIRES_DIFFERENT_PERSON=true`). The lesson inherits the **most restrictive intersection of ACL tags** from the source evidence, is scrubbed by `PIIRedactor`, supersedes prior lessons without deleting history, upserts only text-free vectors to Pinecone, and is tagged with `source_type: "past_decision_lesson"`. A single outcome gives **zero confidence boost** to future hypotheses; aggregate calibration statistics in `evals/calibration_eval.py` (Brier score, Expected Calibration Error) require at least `MIN_OUTCOMES_FOR_STATS=10` recorded outcomes.

Phase 8: Next.js 15 Frontend — ALBACONNECT Brand System, Confidential Cookie Proxy & CJK Diffing

Binding ALBACONNECT Brand Tokens & Zero-Green Status Design

Phase 8 builds the bilingual (`/ja` and `/en`) Next.js 15 App Router Decision Workspace in `apps/web/`, adhering strictly to the official ALBACONNECT Brand Guidelines codified in `apps/web/tailwind.config.ts` and documented in `docs/DESIGN_DECISIONS.md` and `docs/I18N_REVIEW.md`:

- **Color Scales (Light Theme Only, Zero Gradients):** Primary 40 `#0052FF` (primary interactive actions), Secondary 40 `#006685` (evidence citations, structure, and verified completion states), Tertiary 40 `#8424DA` (subtle accent for AI-generated hypotheses), Quaternary 40 `#B6250A` (alerts, blocks, and compliance warnings), and Neutral `#1B1B1B` to `#5E5E5E` with hairline rules `#C6C6C6` / `#E2E2E2`.
- **Why There Is Zero Green:** The ALBACONNECT brand palette intentionally contains no green hue. Rather than inventing an off-brand green for "completed/approved" badges, ALBA ONE renders completion states using an explicit check icon paired with a bilingual text label in Neutral or Secondary (`#006685`), tested by `apps/web/tests/design-tokens.test.ts` to guarantee zero prohibited hex codes appear anywhere in `apps/web/src/`.
- **Typography:** Pairs `Zen Kaku Gothic New` for Japanese prose, `Manrope` with `font-variant-numeric: tabular-nums` for Latin copy and aligned financial/SLA tables, and `Inter Tight` for large English display headings.

Confidential Route-Handler Proxy, Character-Level Japanese Diffing & Mock Ban

Browser JavaScript never touches bearer tokens or calls FastAPI directly. Instead, `apps/web/src/app/api/proxy/[...path]/route.ts` reads the `httpOnly; Secure; SameSite=Lax` session cookie (`alba_session`) on the server and forwards it as an `Authorization: Bearer ...` header to FastAPI while preserving unbuffered `ReadableStream` bodies for Server-Sent Events (`text/event-stream`) with `Last-Event-ID` resume support (`apps/web/src/lib/api/sse.ts`).

When an approver edits conditions under `approve_with_conditions` in `StickyActionBar.tsx`, `apps/web/src/lib/diff.ts` automatically selects **character-level Longest Common Subsequence (LCS) diffing for Japanese** (since Japanese lacks spaces between words) and word-level diffing for English. Finally, to guarantee that development mock fixtures (`apps/web/src/lib/api/mocks.ts`) can never ship to production, `apps/web/scripts/assert-no-mocks-in-prod.mjs` fails `npm run build` if `NODE_ENV=production` and `NEXT_PUBLIC_USE_MOCKS=true`, and `apps/web/scripts/check-startup.mjs` inspects `.next/required-server-files.json` at container boot before starting `server.js`.

Phase 9: Observability, Versioned USD/JPY Cost Accounting & 11-Family Red-Team Harness

PostgreSQL as System of Record for Spans (`ADR-0004` & `0002_observability_spans.py`)

In enterprise operations, the Quality UI and operational metrics endpoints (`GET /api/v1/metrics/decisions/{id}`, `GET /api/v1/metrics/summary`, `GET /api/v1/evals` in `api/metrics.py`) must remain 100% functional even if LangSmith is unreachable or disabled by tenant policy. Phase 9 (justified in `docs/adr/0004-unified-spans-table.md`) introduces a unified `spans` table and a per-run rollup table `decision_run_metrics` in PostgreSQL alongside asynchronous LangSmith export.

Every LLM call, embedding batch, rerank request, Bedrock guardrail evaluation, MCP tool invocation, and deterministic calculator call writes one row in `spans` (`observability/spans.py`), capturing `span_id`, `parent_span_id`, `trace_id`, `tenant_id`, `decision_id`, `run_id`, `phase_step` (16-value enum from `clarify` to `writeback`), `kind`, `model_or_tool`, `prompt_version`, `guardrail_version`, `model_config_version`, `tool_def_hash`, `tokens_in`, `tokens_out`, `latency_ms`, `ttfb_ms`, and `status`. By default

(`TRACE_CAPTURE_CONTENT=false`, enforced in production by `startup_checks.py`), raw prompt and output text are never stored in `spans`—only SHA-256 hashes (`prompt_hash`, `output_hash`) and byte lengths are persisted.

Versioned USD/JPY Cost Calculation & Offline Cost Recomputation

`apps/api/src/alba_one/observability/cost.py` computes per-span and per-run costs using the versioned price catalog in `config/models.yaml` (`price_table_version: "2026-10-24.v1"`, `fx_usdjpy: 151.45`, `fx_rate_date: "2026-10-24"`). If a model is missing from the price table, `cost_amount` is recorded as `None` and `unpriced_calls` is incremented rather than silently recording `$0.00`. When vendor prices or FX rates are updated, `scripts/recompute_costs.py` recomputes historical span costs deterministically.

11-Family Bilingual Red-Team Harness (`redteam/`) & Dual-Target Meta-Verification

Phase 9 builds a comprehensive adversarial red-team harness covering 11 attack families × 10 scenarios (5 Japanese + 5 English per family = **110 adversarial scenarios** in `redteam/scenarios/catalog.py`), paired with 8 deterministic scorers in `redteam/scorers.py`. All test fixtures use clearly fictional entities (架空テスト海運株式会社, `Fictional Test Maritime Corp`, `FICTIONAL-CODE-KAGUYA-架空`) and per-run random canary tokens (`CANARY-<hex>` generated by `redteam/fixtures/generator.py`). Figure 2 illustrates the 11 attack families, their primary blocking layers, and the production break-glass filtering enforced in Phase 11.

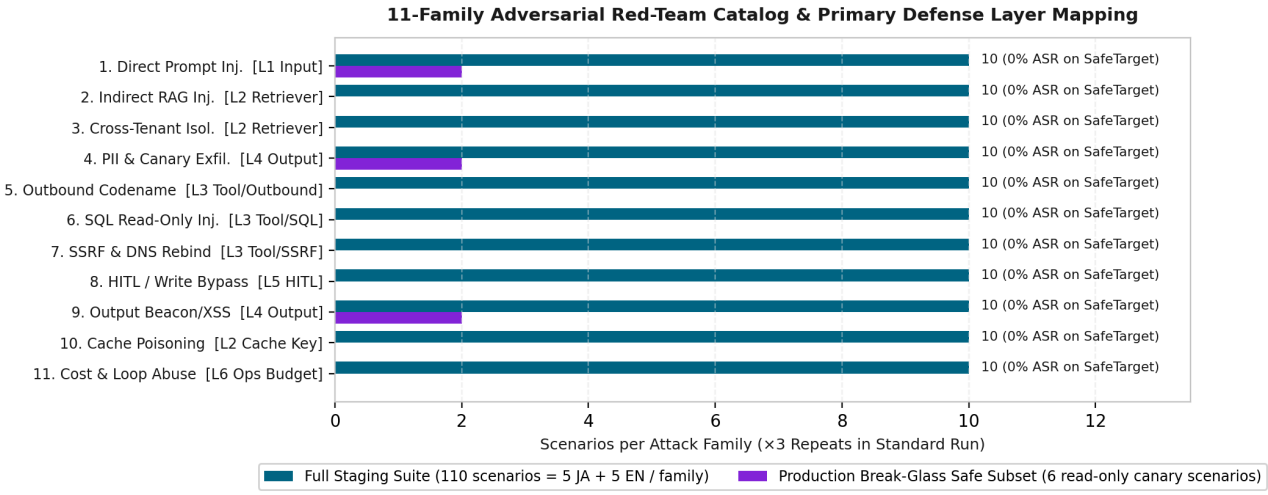


Figure 2: The 11 adversarial attack families (110 total scenarios across Japanese and English), their primary guardrail blocking layers, and the 6-scenario production-safe break-glass subset.

To prove that our deterministic scorers actually catch vulnerabilities and are not tautological pass-stubs, `redteam/targets.py` provides both `SafeTarget` (wired through all 7 guardrail layers) and `DeliberatelyVulnerableTarget` (with guardrails bypassed). Our meta-test in `test_redteam_and_safety.py` verifies that `DeliberatelyVulnerableTarget` fails on every single attack family while `SafeTarget` achieves 0 critical bypasses across 3 repeats. In accordance with Principle 10, reports never claim the system is `"secure"`; instead, `redteam/runner.py` emits the exact factual statement:

> *`"no critical bypass observed in 110 scenarios × 3 repeats on 2026-10-24/1.0.0"`*

Phase 10: Deployment, Hardened Containers, OIDC/JWKS Auth, Fail-Closed Boot & Secret Rotation

Container Hardening, Read-Only Root Filesystems & 4-Tier Network Segmentation

Phase 10 separates local development bootstrap files (`infra/`) from production deployment artifacts (`deploy/`). All three production Dockerfiles—`apps/api/Dockerfile` (with `api` and `worker` targets), `apps/api/Dockerfile.mcp`, and `apps/web/Dockerfile`—enforce multi-stage builds, base images pinned by immutable `@sha256:` digests (refresh procedure documented in `docs/RUNBOOK_DEPLOY.md`), numeric non-root user `USER 10001:10001`, fixed `HOME=/home/alba`, `PYTHONDONTWRITEBYTECODE=1`, removal of package managers (`apt`, `dpkg`, `pip`, `npm`) from final stages, and `read_only: true` root filesystems with explicit writable `tmpfs`/`emptyDir` mounts at `/tmp` and `/app/.next/cache` (verified by `ci/scripts/check_images.sh`).

Network topology is segmented across four isolated zones in both `infra/docker-compose.prod-like.yml` (labeled strictly for local hardening verification, not production deployment) and `deploy/helm/alba-one/templates/networkpolicies.yaml`: (1) `edge` (reverse proxy ↔ web/api, terminating TLS 1.2/1.3 and setting HSTS, CSP, `X-Content-Type-Options: nosniff`, and `X-Frame-Options: DENY` in one place via `deploy/docker/Caddyfile` and Kubernetes Ingress), (2) `app` (web ↔ api ↔ MCP servers/workers with Istio `STRICT` mTLS in `deploy/security/mtls-and-pss.yaml`), (3) `data` (internal-only network for managed RDS PostgreSQL 16 and ElastiCache Redis 7 provisioned via `deploy/terraform/main.tf`), and (4) `egress` (routing external MCP server traffic exclusively through a domain-allowlisted Squid/Envoy egress gateway).

OIDC / JWKS Verification, Anti-Amplification Cache & Dual-Secret S2S Rotation (`core/auth_oidc.py`)

In `apps/api/src/alba_one/core/auth_oidc.py` and `core/tenancy_logic.py`, authentication in staging and production uses cryptographic OIDC JWT verification (`OIDCVerifier` backed by `cryptography` and `PyJWT[crypto]`):

- **Algorithm Allowlist & Claim Verification:** Accepts only asymmetric `RS256` and `ES256` signatures by default; unconditionally rejects `alg: "none"`, all HMAC (`HS*`) algorithms, and ID tokens (`token_use: "id"` or `typ: "id_token"`). Verifies `iss`, `aud`, `sub`, `exp`, `nbf`, and `iat` (rejecting tokens older than `OIDC_MAX_TOKEN_AGE_SECONDS=3600`).
- **Strict Token-Only Claim Mapping:** Extracts `tenant_id`, `roles`, and `lang` exclusively from the verified JWT. Unknown roles are dropped; a missing tenant claim returns HTTP `403`; and any request supplying a conflicting `X-Tenant-Id` header or `tenant_id` query parameter is rejected with HTTP `403` and logged as `security_event_conflicting_tenant_id`.
- **JWKS Anti-Amplification & Stale Fallback (`JWKSCache`):** Fetches JWKS strictly over HTTPS outside development with a 3.0s timeout and 64 KiB cap. On an unknown `kid`, refreshes at most once per `JWKS_MIN_REFRESH_INTERVAL_SECONDS=30` to prevent random-`kid` DoS floods. If the IdP is unreachable during refresh, serves cached keys up to `JWKS_MAX_STALE_SECONDS=3600` while incrementing `stale_fallback_alert_count`, then fails closed with HTTP `503`.
- **Zero-Downtime S2S Token Rotation (`InternalBearerRotator`):** API-to-MCP internal tokens accept both `INTERNAL_S2S_CURRENT_SECRET` and `INTERNAL_S2S_PREVIOUS_SECRET` during the rotation window (documented in `deploy/runbooks/key-rotation.md`).

Startup Fail-Closed Validator (`core/startup_checks.py`)

Executed in `api/main.py` before the app accepts traffic, `enforce_startup_checks()` treats an unset or unknown `APP_ENV` as `production` and evaluates every rule in one pass—aborting boot with a single aggregated error listing all violations if `AUTH_STUB_ENABLED=true`, OIDC URLs are missing or non-HTTPS, debug/reload/OpenAPI docs are enabled, CORS uses wildcards, secrets match placeholders from `.env.example`, `DATABASE_URL` lacks `sslmode=require|verify-full`, `REDIS_URL` lacks `rediss://` and password auth, `TRACE_CAPTURE_CONTENT=true`, or guardrails are in `monitor` mode without `ALLOW_GUARDRAIL_MONITOR_IN_PROD=true`.

Phase 11: Post-Deployment Red Teaming, 4-Control Isolation, Wilson CI Gates & Safe Containment

Four Independent Target-Isolation Controls (`redteam/isolation.py` & `redteam/ROE.md`)

Because a YAML flag alone could be modified by a mistaken commit, Phase 11 enforces four independent controls before any post-deployment red-team run executes:

1. **Control 1 — Code Allowlist (`redteam/allowed_targets.yaml`):** Refuses any unlisted target host and defaults `allow_production: false`.
2. **Control 2 — Pre-Attack Target Identity Check (`TargetIdentity`):** Queries the target's health/build endpoint before sending any attack. If a target configured as `staging` self-reports `APP_ENV=production` (e.g., DNS misconfiguration or wrong secret), `verify_target_isolation()` aborts immediately with `Control 2 Abort`.
3. **Control 3 — Repository Protection (`.github/CODEOWNERS`):** Requires `@albaconnect/security-owners` approval on `redteam/ROE.md`, `redteam/allowed_targets.yaml`, `redteam/quarantine.yaml`, and `.github/workflows/redteam-*.yml`, with scheduled runs (`redteam-scheduled.yml`) restricted to `refs/heads/main` on CI runners outside the production cluster.
4. **Control 4 — Break-Glass Production Gate & `production_safe` Filtering:** Production probes (`redteam-breakglass-prod.yml`) require manual `workflow_dispatch` inputs (`justification`, `ticket_id`, `time_window_minutes`, `scope`), approval in the `production-probe` GitHub Environment, `REDTEAM_PROD_ENABLED=1`, `--production-authorised`, an active time window, and `filter_scenarios_for_environment(..., is_production=True)`—which restricts execution to the 6 read-only canary scenarios marked `production_safe: true` against `tenant_fictional_canary`.

Baseline Diffing, Wilson 95% Confidence Intervals & Blast-Radius Containment (`redteam/compare.py`, `incidents.py`)

Every run records audit rows in `redteam_runs` and `redteam_results` (migration `0003_redteam_runs_and_results.py`). In `redteam/compare.py`, candidate runs are compared against the most recent passing baseline sharing the same `scenario_set_hash`, `dataset_hash`, and `scorer_version` (baselines move only via reviewed PRs). To prevent statistical noise on small samples from triggering false over-refusal gates, `wilson_confidence_interval(k, n)` computes exact 95% Wilson score bounds per language (`ja` and `en` separately):

$$CI_{95\%} = (\hat{p} + z^2/2n \pm z\sqrt{[\hat{p}(1-\hat{p})/n + z^2/4n^2]}) / (1 + z^2/n) \quad \text{(for } k=50, n=100\text{: } (0.403832, 0.596168)\text{)}$$

When a critical bypass occurs (`critical_bypass_count > 0`), Argo Rollouts aborts canary promotion automatically via `deploy/observability/argo-rollout-analysis.yaml`, failing scenarios are re-run `R` times to measure reproduction rate, hostile prompt/output excerpts are escaped and truncated inside a warning block by `sanitize_untrusted_for_incident()`, and `evaluate_containment()` recommends containment actions ranked by blast radius—**never auto-triggering `OPS_KILL_SWITCH=true`** so a flaky scorer cannot cause a multi-tenant outage.

Conclusions, Operational Readiness & Reference Index

Across Phases 1 through 11, ALBA ONE (意思決定OS) demonstrates that an enterprise AI system can combine frontier bilingual reasoning with strict, provable operational controls. By executing domain calculations, multi-criteria comparisons, and verdict ceiling checks in deterministic Python; isolating on-premise text from cloud vector stores; enforcing seven layers of guardrails; binding approvals to OIDC-verified human identities; and validating defenses continuously through a 4-control isolated red-team harness, ALBACONNECT Inc. achieves a production-ready architecture aligned with ISO/IEC 27001 ISMS governance.

Before cutting over live production traffic, the repository owner completes the explicitly tracked sign-offs cataloged in `docs/STUBS.md`: generating lockfiles (`uv lock` and `npm install`) on a networked build host, signing off regulatory and criterion weights in `config/regulatory_params.yaml` and `config/decision_criteria/*.yaml`, transitioning golden datasets from `"draft"` to `"reviewed"` in `goldens/**/*.json`, tuning container and RDS/ElastiCache sizing from load tests, and enabling the documented GitHub branch-protection and `production-probe` Environment reviewer rules.

Production Cutover & Owner Sign-Off Checklist

Checklist Item & Artifact Location	Required Owner Action Prior to Live Production Cutover	Verification Command / Gate
1. Lockfiles & Dependency Pinning (`uv.lock`, `apps/web/package-lock.json`)	Generate deterministic Python 3.12 and Node.js 22 lockfiles on a networked build host and commit to `main`.	`uv lock && uv sync --frozen`
2. Regulatory & Criteria Sign-Off (`config/regulatory_params.yaml`, `config/decision_criteria/*.yaml`)	Replace `verified_by: TODO(owner)` and `weights_reviewed_by: TODO(owner)` with legal/domain owner sign-off to unlock full `recommend_approve` verdicts.	`pytest apps/api/tests/unit/test_judge_comparison_and_gates.py`
3. Golden Dataset Review & Thresholds (`goldens/**/*.json`, `evals/thresholds.yaml`)	Promote reviewed bilingual golden items from `"draft"` to `"reviewed"` and set calibrated pass/fail thresholds so CI runs without `--allow-draft`.	`python3 evals/retriever_eval.py --compare`

Checklist Item & Artifact Location	Required Owner Action Prior to Live Production Cutover	Verification Command / Gate
4. GitHub Branch Protection & Environments (`.github/CODEOWNERS`, `docs/SECURITY.md`)	Enable `CODEOWNERS` review on `main` and configure required human reviewers on the `production-probe` GitHub Environment.	`bash scripts/verify_local.sh`

Reference & Repository Artifact Index

- **ALBACONNECT Corporate & Brand Guidelines:** https://albaconnect.co.jp/ and https://albaconnect.co.jp/about/brand (referenced in `apps/web/tailwind.config.ts`, `docs/DESIGN_DECISIONS.md`, and `docs/I18N_REVIEW.md`).
- **Architecture Decision Records (ADRs):** `docs/adr/0001-llm-gateway.md` (Swappable LLM Gateway), `docs/adr/0002-japanese-fts-tokenizer.md` (Sudachi Dual-Mode + `pg_bigm` Japanese FTS), `docs/adr/0003-mem0-vector-store.md` (Self-Hosted Mem0 OSS on PostgreSQL `pgvector`), and `docs/adr/0004-unified-spans-table.md` (Unified PostgreSQL `spans` Table).
- **Security, Guardrails & Deployment Runbooks:** `docs/SECURITY.md`, `docs/GUARDRAIL_COVERAGE.md`, `docs/DATA_FLOWS.md`, `docs/FAILURE_POLICY.md`, `docs/KB_EXTRACTION.md`, `docs/MCP_CATALOG.md`, `docs/RUNBOOK.md`, `docs/RUNBOOK_DEPLOY.md`, `deploy/README.md`, `deploy/runbooks/key-rotation.md`, `redteam/ROE.md`, and `docs/STUBS.md`.